

Documentazione del Sistema

Casa della Salute

Progetto di Ingegneria del Software

Corso di Laurea in Bioinformatica

Università degli Studi di Verona

Barichello Rebecca (VR471455)

Kaur Taniya (VR481429)

Magrograssi Federico (VR472578)

Contents

1 Introduzione

Il sistema software in questione è stato creato per la gestione dei servizi erogati da una Casa della Salute, come visite mediche, prelievi e medicazioni.

Si suppone che il software venga distribuito ai pazienti e al personale della Casa interessati, e che sia usufruibile tramite appositi terminali presenti nella struttura.

Gli utenti, per poter usufruire dei servizi, dovranno essere in possesso delle loro credenziali. Queste verranno generate dal personale di segreteria e comunicate agli utenti stessi.

Obiettivi del sistema

Il sistema è progettato per gestire le attività di una Casa della Salute, dove operano più medici di base, supportati da infermieri per medicazioni e prelievi, e un ufficio di segreteria comune. La struttura ospita:

- 3 ambulatori per visite per adulti, 2 ambulatori per prelievi e medicazioni ad adulti,
- 1 ambulatorio pediatrico,

Funzionalità principali

Il sistema permette di:

- Registrare e gestire i dati di medici, infermieri e pazienti.
- Gestire le prenotazioni per visite, inviando mail di conferma.
- Mantenere una storia delle visite e delle prestazioni effettuate.
- Fornire resoconti annuali delle prestazioni erogate ai pazienti.

Autenticazione e accesso

Il sistema permette l'autenticazione ai pazienti tramite le credenziali fornite dalla segreteria. Il personale medico e infermieristico, invece, può accedere tramite le proprie credenziali personali, sempre nella sezione login.

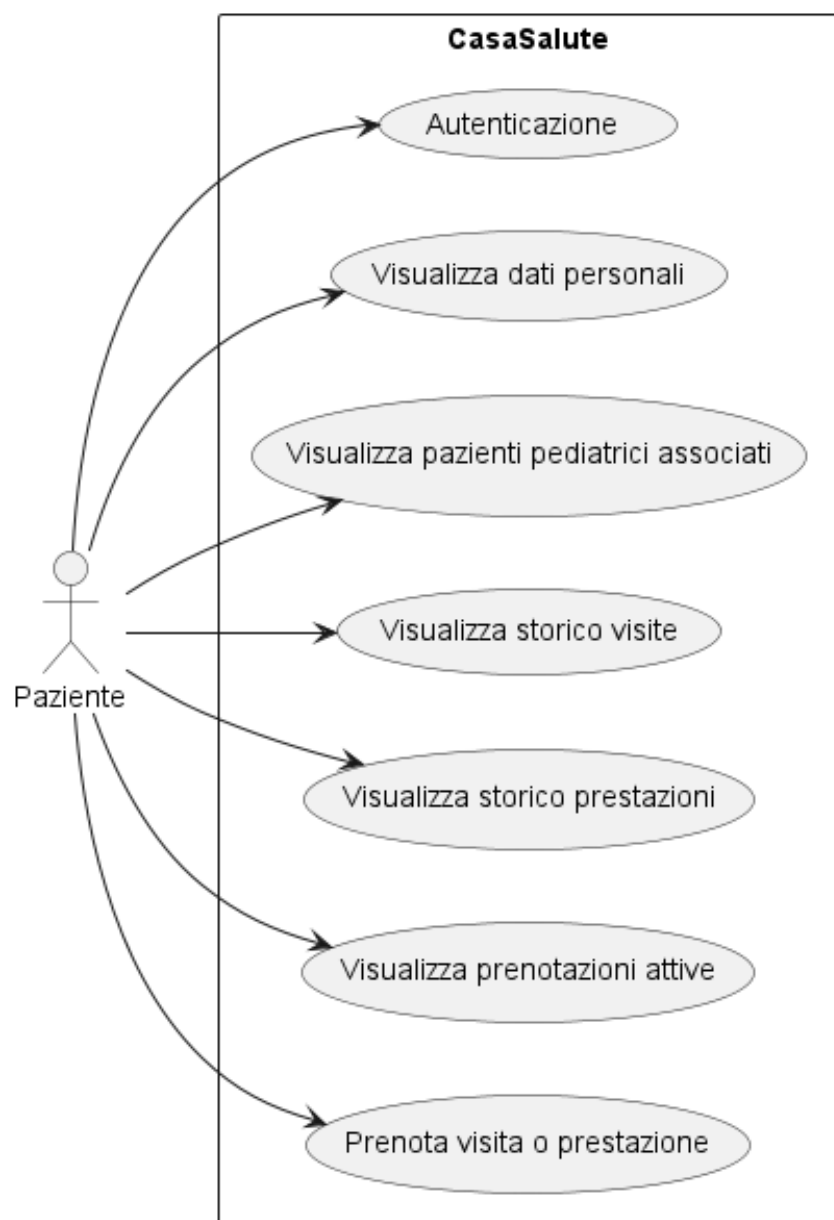
2 Casi d'uso

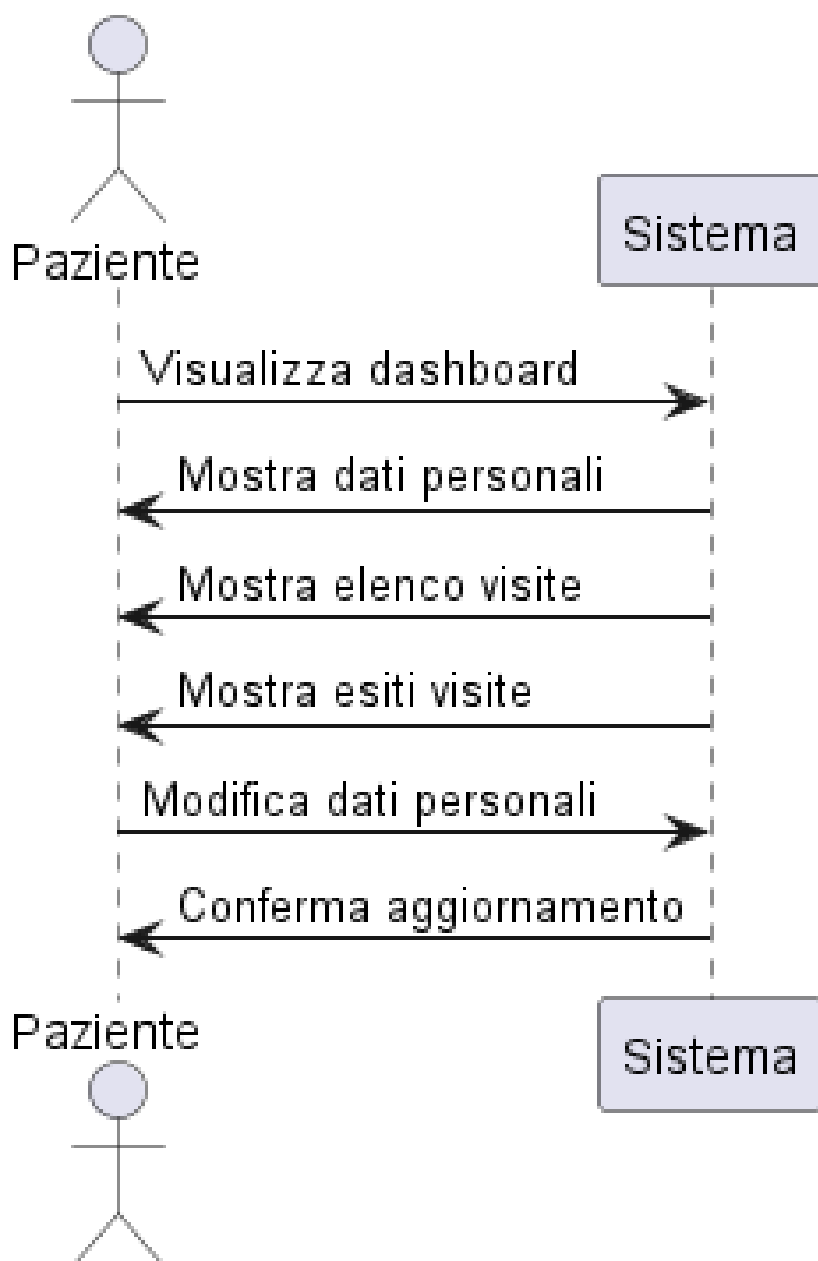
In questa sezione vengono presentati i diagrammi dei casi d'uso per ciascuna categoria di utente che interagisce con il sistema della Casa della Salute. Per ogni figura (paziente, medico, infermiere e segreteria) sono illustrate, tramite due diagrammi, le principali funzionalità accessibili e le interazioni previste con il sistema informatico.

2.1 Paziente

Il paziente può accedere alla propria area personale tramite autenticazione. Una volta autenticato, ha la possibilità di visualizzare la propria dashboard, consultare i dati personali, le visite programmate e gli esiti delle visite già effettuate. Inoltre, può aggiornare i propri dati anagrafici.

Il primo diagramma rappresenta in maniera sintetica i casi d'uso del paziente all'interno del sistema, evidenziando le funzioni principali. Il secondo schema mostra più nel dettaglio le interazioni sequenziali tra il paziente e il sistema.

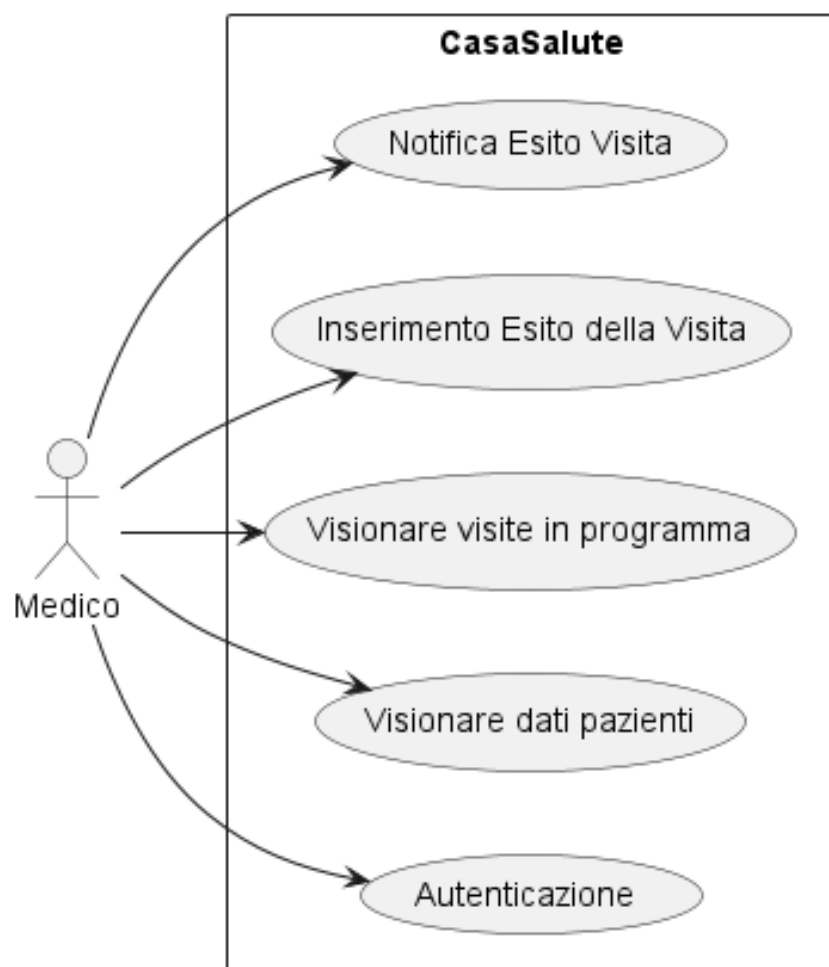


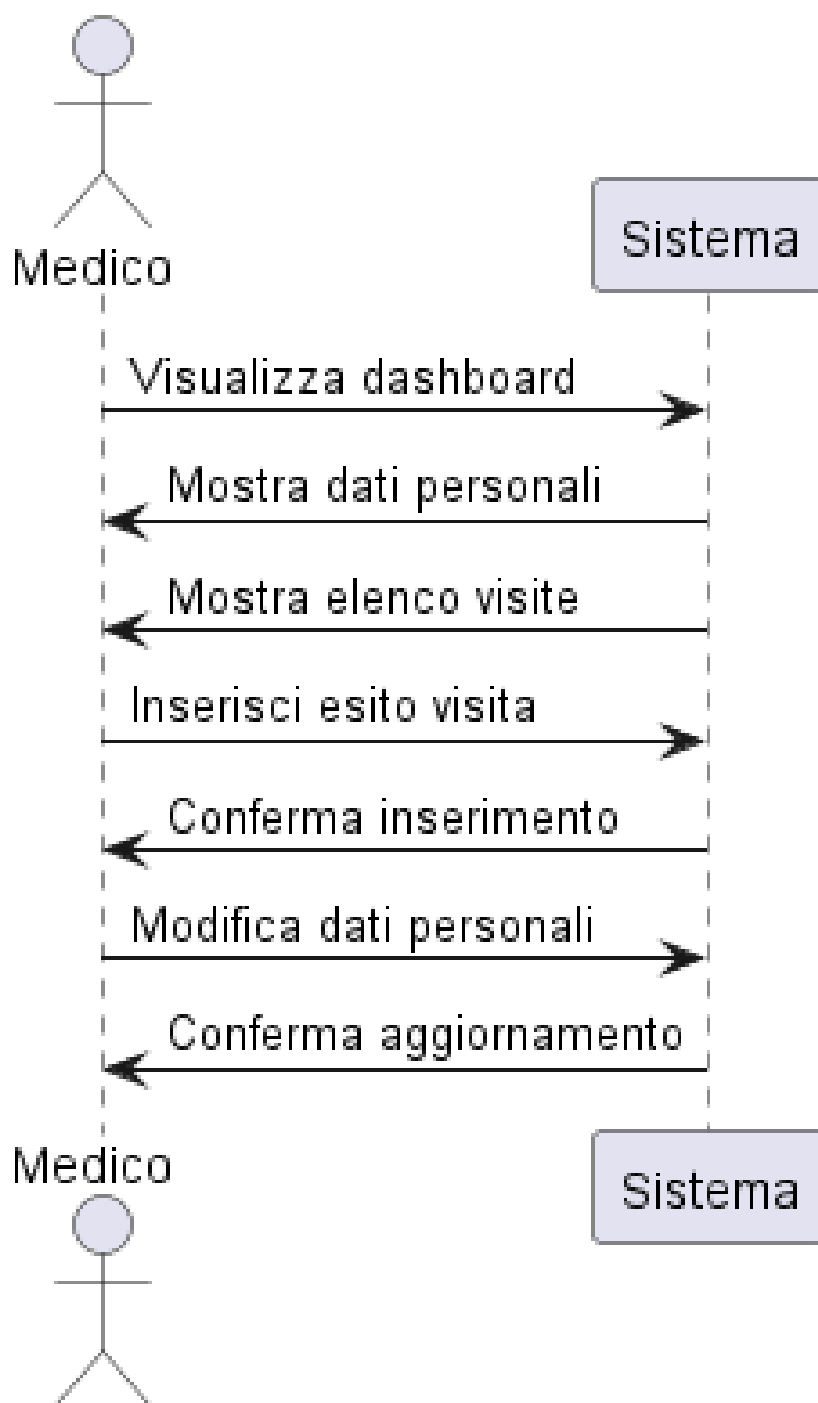


2.2 Personale medico

Il personale medico, dopo aver effettuato l'autenticazione, può consultare la propria area personale per visualizzare l'elenco delle visite in programma e accedere ai dati clinici dei pazienti. Ha inoltre la possibilità di inserire l'esito delle visite effettuate, operazione che comporta anche l'invio automatico di una notifica al paziente interessato. Il medico può infine modificare i propri dati personali.

Nel primo diagramma sono riassunti i principali casi d'uso del medico. Il secondo schema mostra nel dettaglio l'interazione tra il medico e il sistema per ciascuna funzionalità.

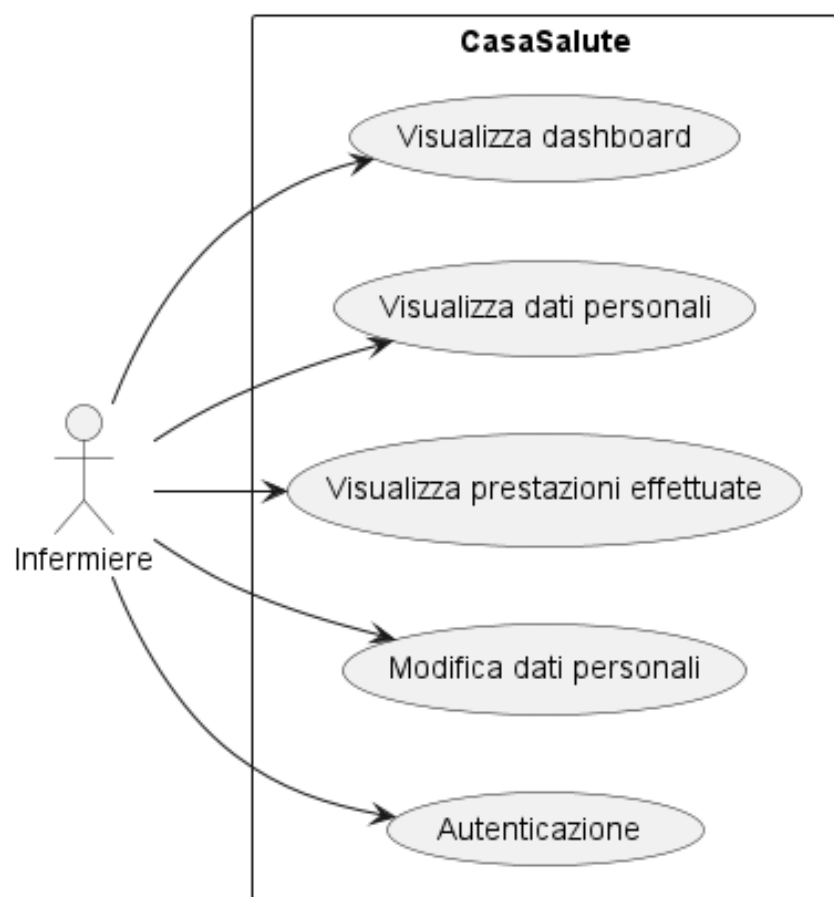


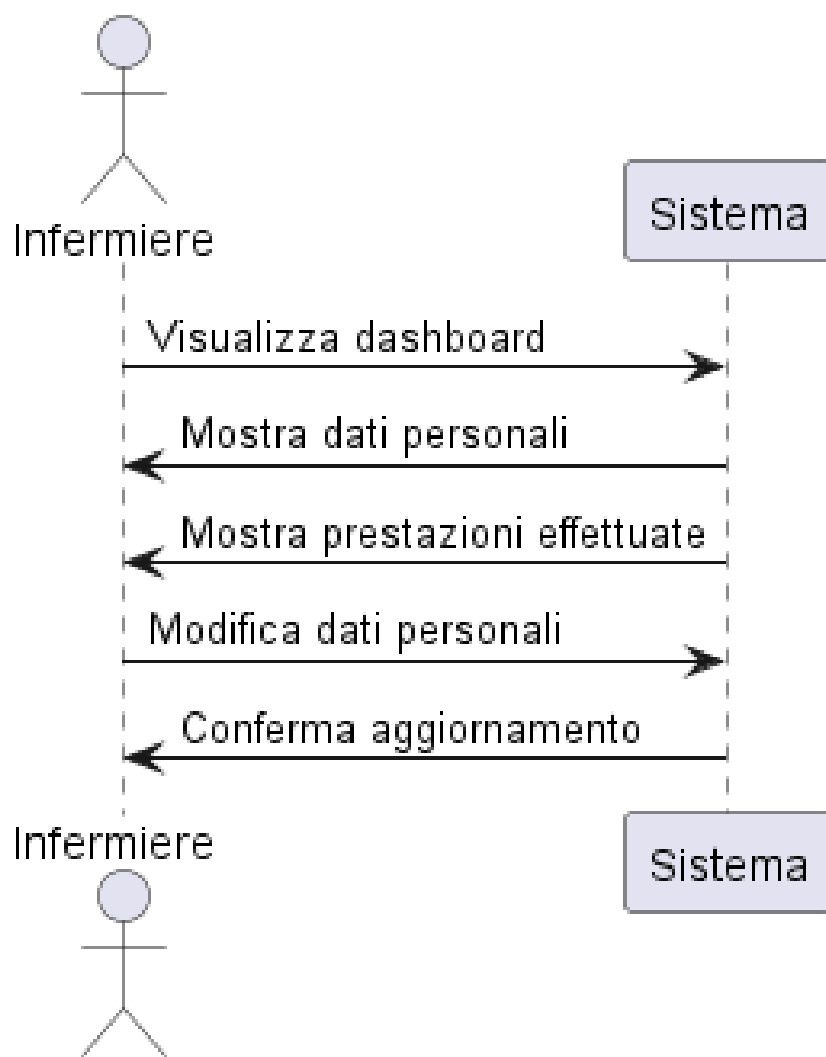


2.3 Infermiere

L'infermiere può accedere alla propria area riservata tramite autenticazione. Una volta entrato nel sistema, ha la possibilità di visualizzare la dashboard personale, i propri dati anagrafici e l'elenco delle prestazioni effettuate. Inoltre, può modificare i propri dati.

Il primo diagramma mostra una panoramica dei casi d'uso principali relativi alla figura dell'infermiere, mentre il secondo fornisce una rappresentazione dettagliata delle interazioni con il sistema.

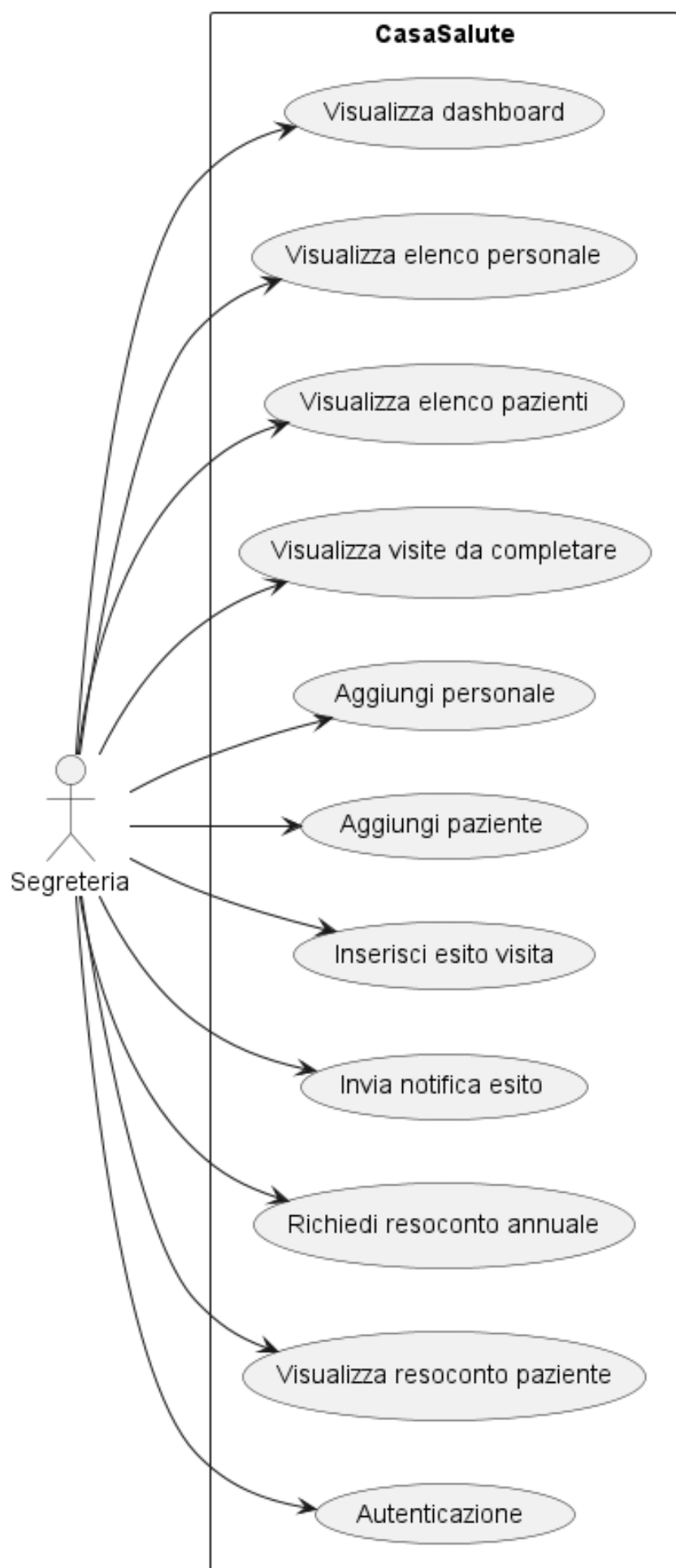


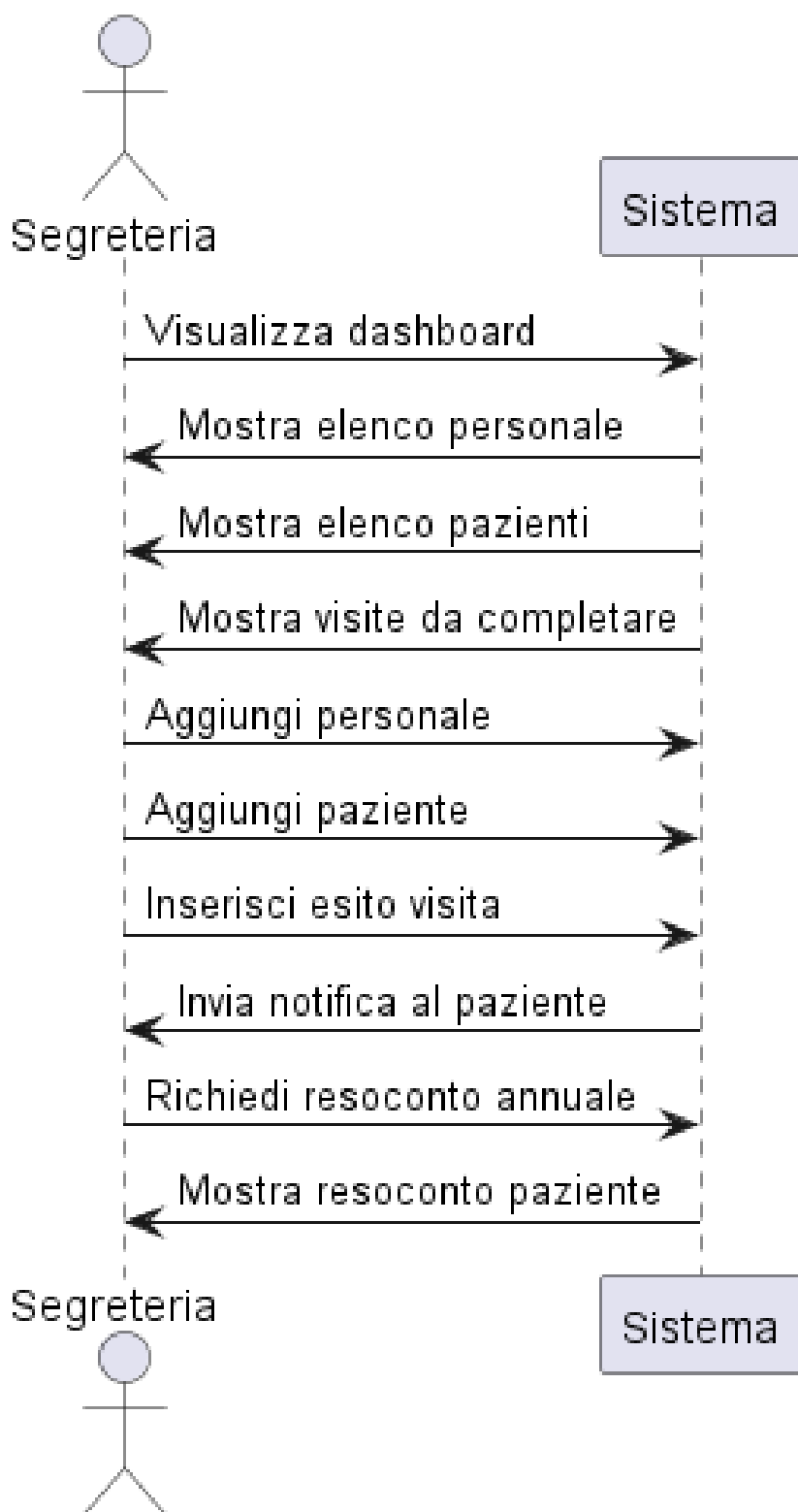


2.4 Segreteria

La segreteria ha accesso a numerose funzionalità gestionali. Dopo l'autenticazione, può visualizzare la dashboard che fornisce una panoramica delle attività da svolgere. Può consultare l'elenco del personale e dei pazienti registrati, visualizzare le visite da completare, e aggiungere nuovi operatori sanitari e pazienti. È responsabile dell'inserimento degli esiti delle visite, dell'invio delle relative notifiche ai pazienti, nonché della generazione dei resoconti annuali delle attività.

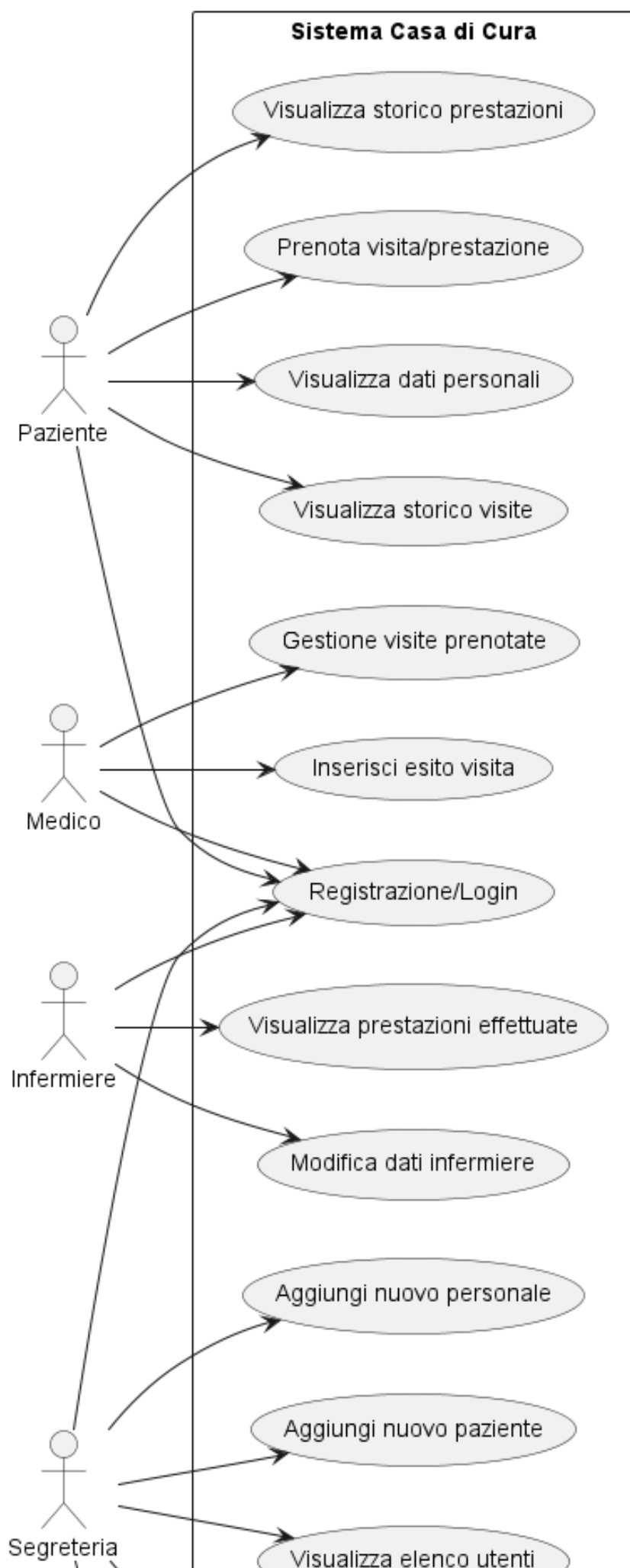
Il primo diagramma riepiloga le principali funzioni disponibili per la segreteria, mentre il secondo approfondisce le specifiche interazioni tra l'utente e il sistema.





2.5 Tutti gli utenti della Casa della Salute

Il diagramma complessivo dei casi d'uso permette di visualizzare l'intero insieme di funzionalità del sistema, mettendo in relazione le quattro categorie di utenti con i rispettivi casi d'uso. Questa visione d'insieme è utile per comprendere la copertura funzionale dell'applicazione.



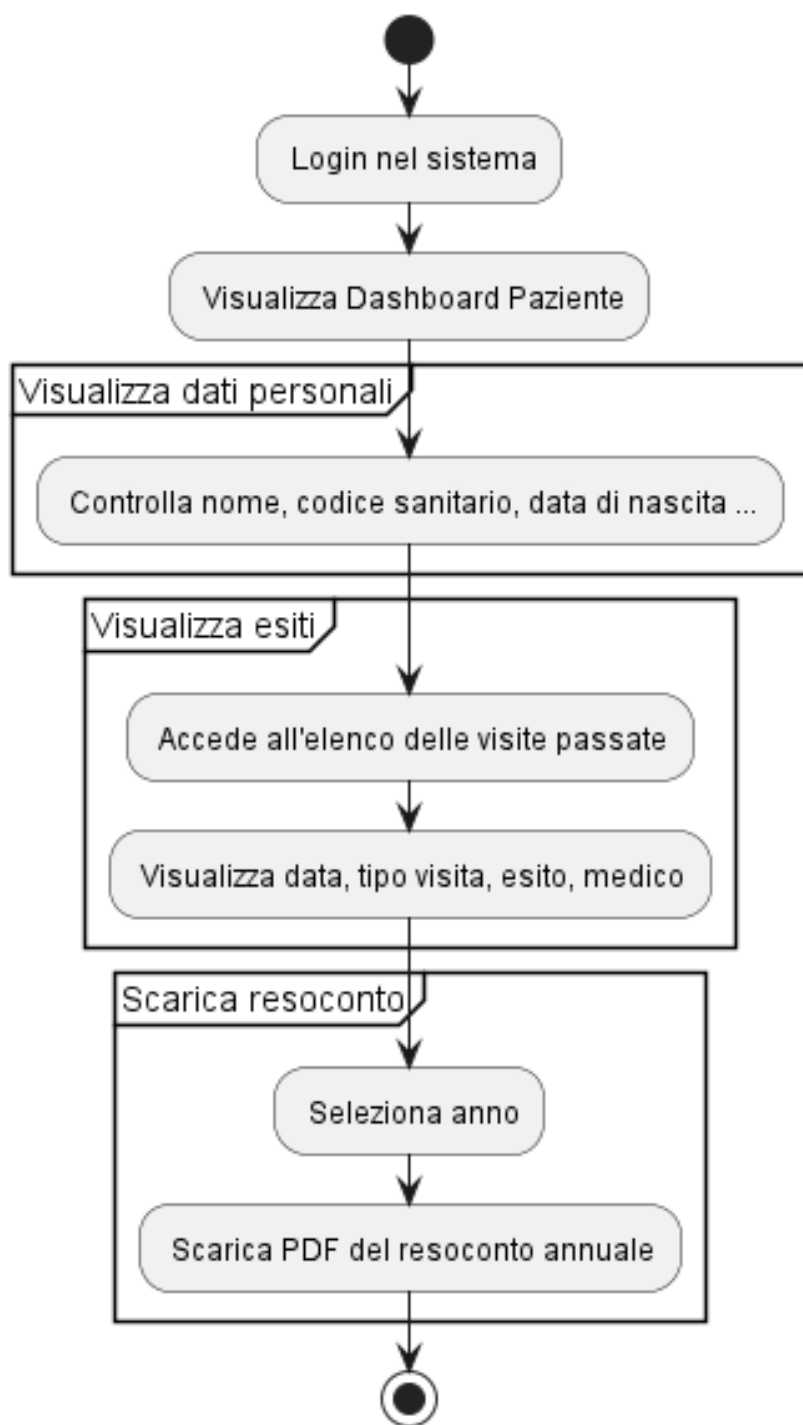
3 Diagrammi delle attività

Come per i diagrammi visti in precedenza, si assume che le operazioni possano essere annullate in qualsiasi momento, ad esempio tramite il logout dal sistema.

Per questioni di leggibilità, si omette la possibilità di visualizzazione e modifica del profilo da parte degli utenti.

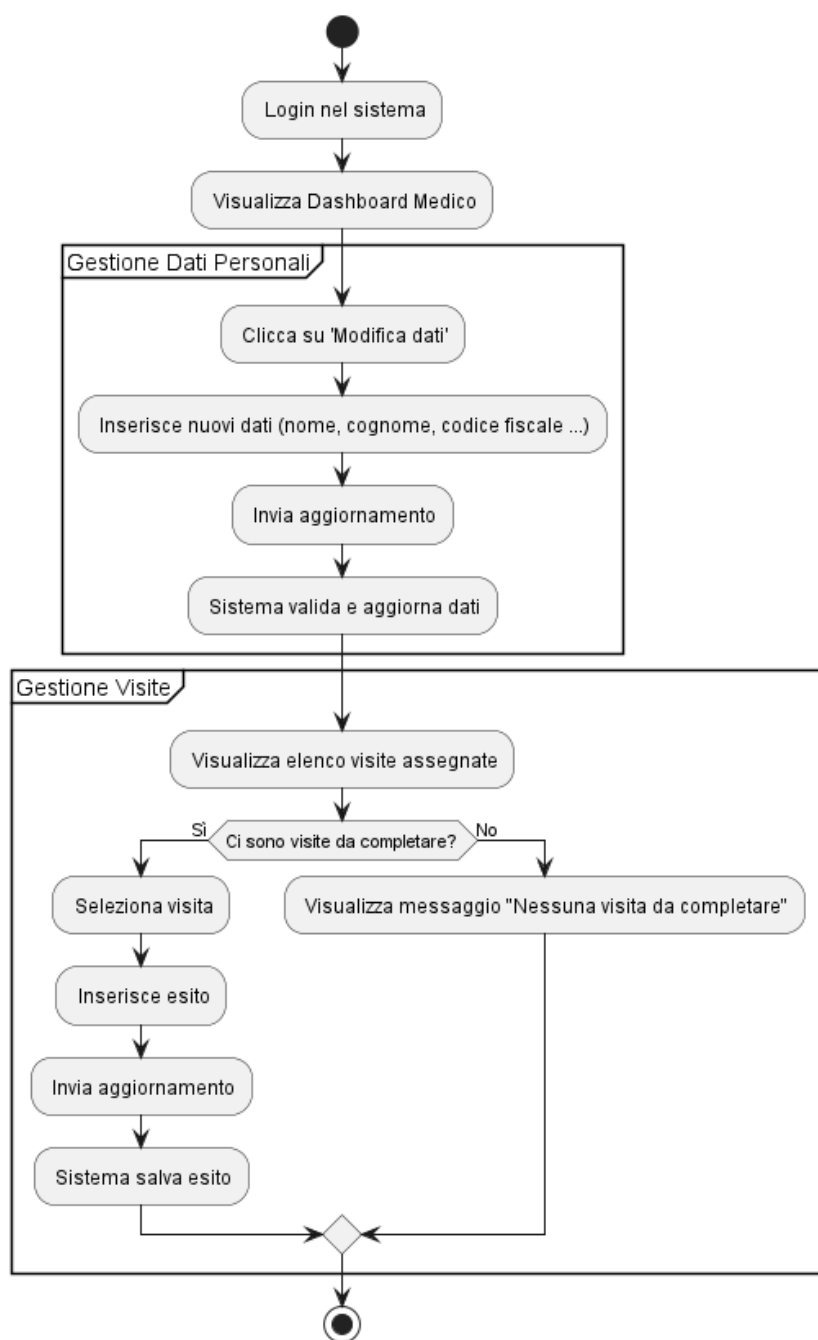
3.1 Paziente

Il diagramma delle attività del paziente descrive il flusso principale delle operazioni che l'utente può compiere una volta autenticato nel sistema. Il paziente può accedere alla propria area personale e scegliere tra diverse azioni: prenotare una visita o una prestazione (come prelievi o medicazioni), consultare l'esito delle visite effettuate, o visualizzare lo storico delle prestazioni ricevute. Ogni azione porta a un completamento specifico, oppure può essere interrotta o annullata in qualsiasi momento.



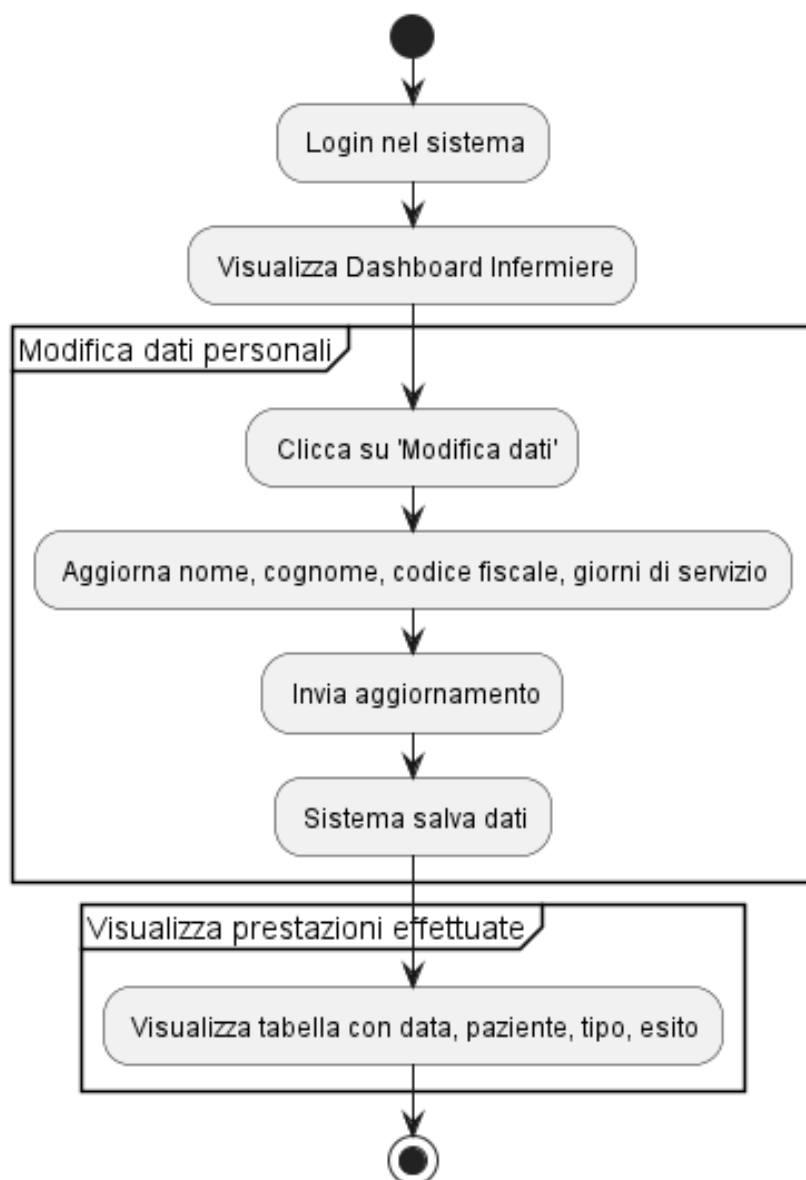
3.2 Personale medico

Il diagramma delle attività del medico mostra il flusso operativo seguito da questa figura professionale. Dopo l'autenticazione, il medico può visualizzare l'elenco delle visite programmate e i dati clinici dei propri pazienti. Può quindi inserire l'esito di una visita appena effettuata, azione che porta automaticamente anche all'invio di una notifica al paziente. L'intero processo è rappresentato in modo sequenziale e chiarisce come si sviluppa la gestione delle visite da parte del personale medico.



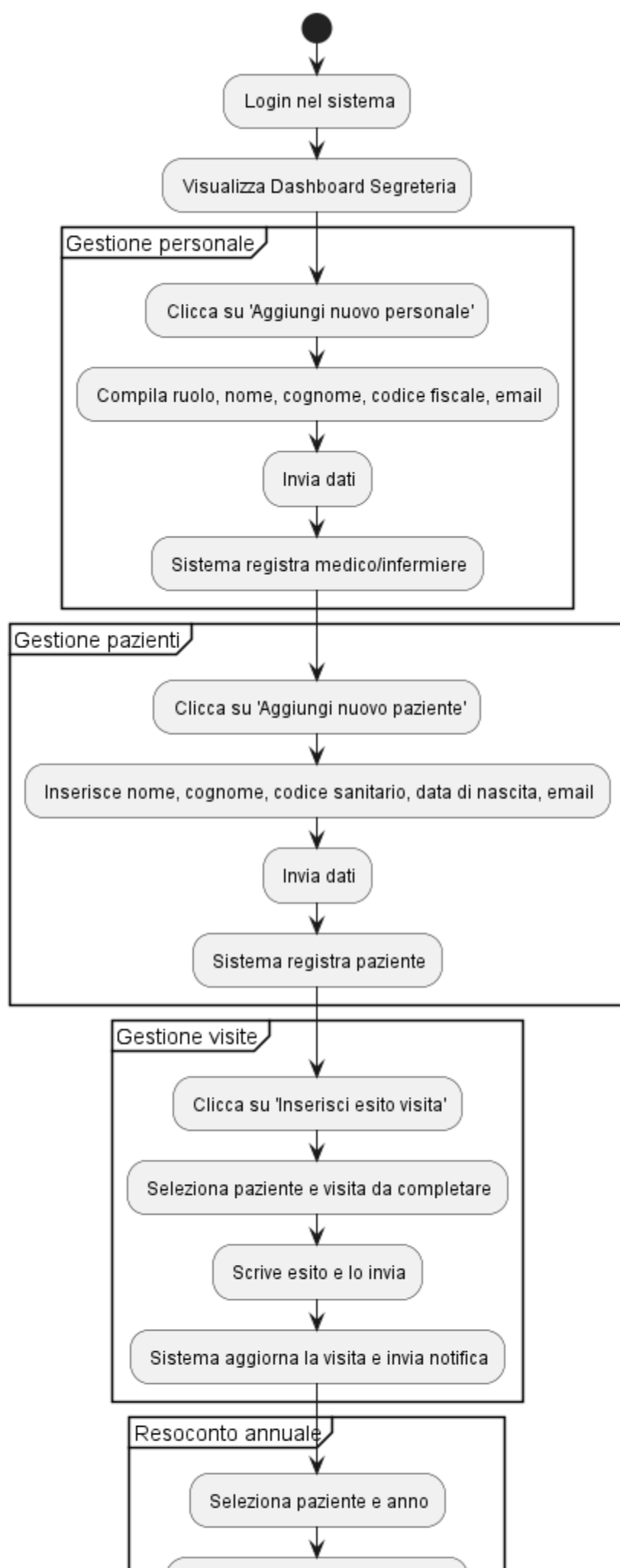
3.3 Infermiere

Il diagramma delle attività relativo all'infermiere mostra le principali operazioni disponibili dopo l'autenticazione. L'utente può accedere alla dashboard per consultare i propri dati anagrafici e l'elenco delle prestazioni precedentemente eseguite. Il flusso si concentra sulla consultazione e sulla possibilità di modificare le informazioni personali, mantenendo il diagramma semplice e focalizzato sulle attività specifiche di questa figura professionale.



3.4 Segreteria

Il diagramma delle attività della segreteria descrive l'intero processo operativo, a partire dall'autenticazione dell'utente. Una volta effettuato l'accesso, la segreteria può visualizzare la dashboard con una panoramica generale. Da qui è possibile consultare gli elenchi di pazienti e personale sanitario, aggiungere nuovi utenti al sistema, inserire gli esiti delle visite e generare report annuali. Inoltre, la segreteria può gestire la programmazione e la modifica delle visite. Il diagramma evidenzia chiaramente l'ampiezza delle funzioni amministrative assegnate a questa figura.



4 Sviluppo: progetto dell'architettura ed implementazione del sistema

Note sul processo di sviluppo

Il processo di sviluppo ha seguito un approccio agile e incrementale. Si assume che solo i pazienti adulti possano essere registrati dal personale di segreteria, mentre i pazienti pediatrici sono inseriti da pazienti maggiorenni.

Gli adulti che non sono maggiorenni non possono avere pazienti pediatrici a loro carico. Quando un paziente pediatrico compie 14 anni, e diventa un ex paziente pediatrico.

Il pediatra può visualizzare gli appuntamenti degli ex pazienti pediatrici e aggiornare l'esito delle visite.

Note sul paziente maggiorenne

Un paziente maggiorenne deve obbligatoriamente essere associato a un medico per poter essere registrato a sistema. Questa scelta è legata alla natura prototipale del progetto. Il paziente maggiorenne potrà poi inserire un paziente pediatrico a suo carico

5 Modelli architetturali e progettazione ad alto livello

Il nostro progetto è stato sviluppato utilizzando il framework Django, che si basa su un'architettura denominata **Model-View-Template (MVT)**. Questo modello è una variante del più noto pattern MVC, adattata alle caratteristiche del framework stesso. Grazie a questa struttura, è possibile separare in modo netto la logica di gestione dei dati, l'elaborazione delle richieste e la visualizzazione lato utente.

4.2.1 Il pattern Model-View-Template (MVT)

Nel pattern MVT, ogni componente ha un ruolo specifico:

- **Model:** definisce la struttura dei dati e la logica di accesso al database. In Django, i model sono scritti in Python all'interno dei file `models.py`, e corrispondono direttamente alle tabelle nel database.
- **View:** contiene la logica dell'applicazione. Si occupa di gestire le richieste HTTP, interagire con i model, elaborare i dati e restituire una risposta, generalmente tramite un template HTML. Le view sono definite nel file `views.py`.
- **Template:** rappresenta la parte visibile all'utente. I template sono file HTML dinamici che permettono di visualizzare i dati elaborati dalle view, usando il linguaggio di templating integrato in Django.

In sintesi, i dati vengono gestiti dai model, elaborati dalle view e presentati all'utente tramite i template. Questo approccio ci ha permesso di organizzare il codice in modo modulare e facilmente manutenibile.

Nel nostro progetto, abbiamo creato diversi modelli per rappresentare utenti, medici, pazienti, prenotazioni e prestazioni. Ogni funzionalità è stata sviluppata come view associata a uno specifico URL, e i dati sono stati presentati attraverso template responsivi.

4.3 Design pattern utilizzati

Durante lo sviluppo abbiamo fatto uso di alcuni design pattern che Django supporta nativamente o che si adattano bene alla sua struttura:

- **Singleton**: questo pattern viene utilizzato in contesti in cui serve garantire un'unica istanza condivisa, ad esempio nella gestione centralizzata delle impostazioni di configurazione.
- **Observer**: Django include un sistema di segnali, come `post_save` e `pre_delete`, che ci ha permesso di eseguire azioni automatiche al verificarsi di determinati eventi (come l'invio di notifiche dopo la prenotazione o la modifica di un esito).
- **Adapter**: in alcuni casi abbiamo integrato librerie esterne adattandole alle esigenze del progetto, in particolare per la validazione dei form e per la gestione dell'interfaccia utente.

4.4 Librerie di terze parti utilizzate

Oltre ai moduli standard di Django e Python, abbiamo utilizzato alcune librerie esterne per migliorare l'efficienza nello sviluppo e la qualità del software:

- **Django**: il framework principale, utilizzato per costruire l'intera architettura web.
- **python-dotenv**: per la gestione sicura delle variabili d'ambiente.
- **crispy-forms**: per migliorare la resa grafica e la personalizzazione dei form HTML.
- **pytest-django**: per scrivere e gestire i test automatici in modo semplice ed efficace.
- **coverage.py**: per valutare la copertura del codice da parte dei test e individuare le parti non ancora verificate.

Versioning del progetto

Durante lo sviluppo del progetto si è utilizzato `git` come sistema di versionamento. Il repository è stato ospitato su GitHub, permettendo una gestione centralizzata del codice sorgente e facilitando la collaborazione tra i membri del team. Ogni modifica al codice è stata registrata tramite commit, e sono stati creati branch per le funzionalità in fase di sviluppo. Questo approccio ha garantito una storia chiara delle modifiche e ha facilitato il processo di revisione del codice.

<https://github.com/Taniyakaur/Project>

6 Installazione e configurazione del sistema

Questa sezione descrive i passaggi necessari per installare e configurare il progetto localmente su una macchina con sistema operativo Windows, macOS o Linux.

Requisiti

Per poter eseguire correttamente il progetto sono richiesti i seguenti strumenti:

- Python 3.10 o superiore
- pip (Python package installer)
- Git
- Un ambiente virtuale (consigliato: `venv`)

Clonazione del repository

Aprire un terminale e clonare il progetto da GitHub:

```
git clone https://github.com/Taniyakaur/Project cd Project
```

Creazione dell'ambiente virtuale

Per creare e attivare un ambiente virtuale:

- Su Windows: `python -m venv venv venv`
- Su macOS/Linux: `python3 -m venv venv source venv/bin/activate`

Installazione delle dipendenze

Con l'ambiente attivo, installare i pacchetti necessari:

```
pip install -r requirements.txt
```

Configurazione delle variabili d'ambiente

Creare un file `.env` nella root del progetto con le seguenti variabili (personalizzare i valori):

```
DEBUG=True SECRET_KEY = chiave_sigaretissima ALLOWED_HOSTS = localhost, 127.0.0.1
```

Migrazione del database

Applicare le migrazioni iniziali:

```
python manage.py makemigrations python manage.py migrate
```

Creazione di un superutente (opzionale)

Per accedere al pannello di amministrazione:

```
python manage.py createsuperuser
```

Avvio del server di sviluppo

Eseguire il server:

```
python manage.py runserver
```

Il sistema sarà accessibile all'indirizzo `http://127.0.0.1:8000`

Accesso al sistema

Per accedere al sistema, utilizzare le credenziali fornite dalla segreteria per i pazienti, o quelle create per il personale medico e infermieristico.

7 Test e validazione

5.1 Unit test

Abbiamo eseguito test di tipo unitario su diverse componenti del sistema, in particolare sui modelli e sulle view. Per questo abbiamo utilizzato il framework di test integrato in Django, basato su `unittest`, e in alcuni casi `pytest-django` per una maggiore flessibilità.

Gli unit test sono stati scritti all'interno dei file `tests.py` presenti in ciascuna app Django.

Esempi di test effettuati:

- Validazione dei form (es. campi obbligatori, formati email, password, ecc.).
- Verifica della corretta creazione e salvataggio dei modelli nel database.
- Controllo che determinate view richiedano l'autenticazione.
- Verifica della logica di assegnazione del medico e delle prenotazioni.

5.2 Test manuali degli sviluppatori

Abbiamo svolto test manuali sistematici simulando le azioni tipiche degli utenti:

- Login e logout di utenti di diversi ruoli (medico, segreteria, paziente).
- Registrazione nuovi utenti e modifica dei dati anagrafici.
- Prenotazione e cancellazione di prestazioni.
- Inserimento di esiti e verifica della notifica al paziente.
- Gestione di situazioni particolari (es. doppia prenotazione nello stesso orario).

5.3 Copertura del codice e strumenti

Per misurare la copertura dei test automatizzati abbiamo utilizzato `coverage.py`, uno strumento che mostra quali righe di codice sono state eseguite durante i test. Questo ci ha permesso di identificare porzioni di codice non ancora testate.

5.4 Proposte per test futuri

In futuro, sarebbe utile integrare:

- Test end-to-end con **Selenium** o **Playwright**, per simulare l'interazione dell'utente con l'interfaccia web.
- Test di carico e prestazioni con **Locust** o **JMeter**, per verificare il comportamento del sistema sotto stress.

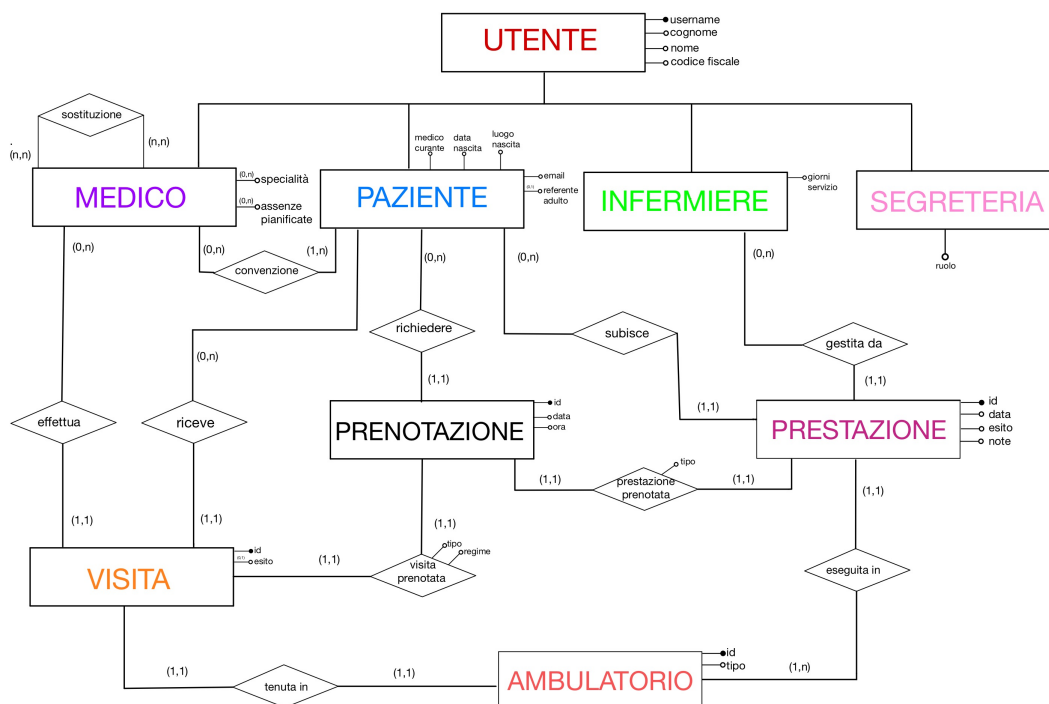
sectionStruttura della base di dati

La base di dati progettata per la gestione della casa di cura si basa su un modello entità-relazione (E-R) ben strutturato, che include le entità principali coinvolte nel processo di erogazione delle prestazioni sanitarie, nonché le relazioni tra esse. Il modello è organizzato secondo una gerarchia centrata sull'entità **Utente**, dalla quale derivano entità specializzate come **Medico**, **Paziente**, **Infermiere** e **Segreteria**.

Entità

- **Utente**: rappresenta un'entità generica dalla quale derivano tutti i soggetti che interagiscono con il sistema. Contiene attributi comuni come `username`, `nome`, `cognome`, `codice fiscale`.
- **Medico**: entità specializzata che può essere soggetta a *sostituzioni* o a *assenze pianificate*, ed è associata a uno o più *pazienti* tramite una *convenzione*.
- **Paziente**: possiede attributi come `data di nascita`, `luogo di nascita`, `email`, `referente adulto` e `email del referente`. Può effettuare una o più *prenotazioni*.
- **Infermiere**: è associato alle *prestazioni* che subisce il paziente, e ha come attributo `giorni di servizio`.
- **Segreteria**: gestisce le *prestazioni* e possiede l'attributo `ruolo`.
- **Prenotazione**: contiene le informazioni relative a `id`, `data`, `ora`, `tipo`, e può essere collegata a una *visita* o a una *prestazione*.
- **Visita**: comprende gli attributi `id`, `esito`, ed è tenuta in un *ambulatorio* da un *medico*.
- **Prestazione**: registra l'attività sanitaria eseguita, con attributi `id`, `data`, `esito`, `note`.
- **Ambulatorio**: rappresenta la struttura fisica in cui avviene una prestazione o una visita. Include `id` e `tipo`.

Lo schema E-R è il seguente:



Modello Relazionale

- **Utente** (username, nome, cognome, codiceFiscale)
- **Paziente** (username, dataNascita, luogoNascita, email, referenteAdulto*, emailReferente*, medicoConvenzionato)
- **Medico** (username, specialità, assenzePianificate)
- **Infermiere** (username, giorniServizio)
- **Segreteria** (username, ruolo)
- **Prenotazione** (id, data, ora, tipo, richiedente)
- **Visita** (id, esito, idPrenotazione, medico, paziente, ambulatorio)
- **Prestazione** (id, data, esito, note, idPrenotazione, infermiere, paziente, ambulatorio)
- **Ambulatorio** (id, tipo)
- **Sostituzione** (medicoSostituto, medicoSostituito)

Vincoli di Integrità Referenziale

[leftmargin=2.5em]

- `Paziente.medicoConvenzionato` → `Medico.username`
- `Paziente.referenteAdulto` → `Paziente.username`
- `Paziente.emailReferente` → `Paziente.email`
- `Prenotazione.richiedente` → `Paziente.username`
- `Visita.idPrenotazione` → `Prenotazione.id`
- `Visita.medico` → `Medico.username`
- `Visita.paziente` → `Paziente.username`
- `Visita.ambulatorio` → `Ambulatorio.id`
- `Prestazione.idPrenotazione` → `Prenotazione.id`
- `Prestazione.infermiere` → `Infermiere.username`
- `Prestazione.paziente` → `Paziente.username`
- `Prestazione.ambulatorio` → `Ambulatorio.id`
- `Sostituzione.medicoSostituto` → `Medico.username`
- `Sostituzione.medicoSostituito` → `Medico.username`

Cardinalità delle Relazioni

Le cardinalità indicano il numero minimo e massimo di occorrenze di una entità che possono partecipare ad una relazione. Sono fondamentali per definire i vincoli logici tra i dati nel modello E-R. Di seguito una sintesi delle cardinalità più rilevanti nel nostro schema:

- **Paziente - Medico:** Ogni paziente è associato ad **uno e un solo** medico (1:1 obbligatoria dal lato paziente), mentre un medico può seguire **più pazienti** (1:N).
- **Paziente - ReferenteAdulto:** Un paziente minorenne può avere un referente adulto. La relazione è **opzionale** per entrambi, ma se presente, è **1:1** tra due pazienti.
- **Prenotazione - Paziente:** Ogni prenotazione è fatta da **un paziente**, ma un paziente può fare **più prenotazioni** (1:N).
- **Visita - Prenotazione:** Una visita è sempre associata ad una sola prenotazione. La relazione è **1:1 obbligatoria**.
- **Prestazione - Prenotazione:** Come per la visita, anche la prestazione è legata ad una sola prenotazione: **1:1 obbligatoria**.
- **Visita - Medico:** Ogni visita è svolta da un solo medico, ma un medico può effettuare più visite: **1:N**.

- **Prestazione - Infermiere:** Simile alla visita: ogni prestazione è svolta da un solo infermiere, che può eseguire più prestazioni: **1:N**.
- **Sostituzione:** Un medico può sostituirne più altri e può essere sostituito da più colleghi: relazione **molti-a-molti (N:M)**.

Tabella Riassuntiva delle Entità

—p3.5cm—p7.5cm—p4cm—

Entità Attributi principali Chiave primaria

Utente CF, username, nome, cognome, password CF

Paziente CF, username, nome, cognome, email, telefono, dataDiNascita, luogoDiNascita, responsabile*, emailResponsabile*, medicoConvenzionato CF

Medico CF, username, nome, cognome, specializzazione, reparto, assenzePianificate CF

Infermiere CF, username, nome, cognome, giorniDiServizio CF

Segreteria CF, username, nome, cognome CF

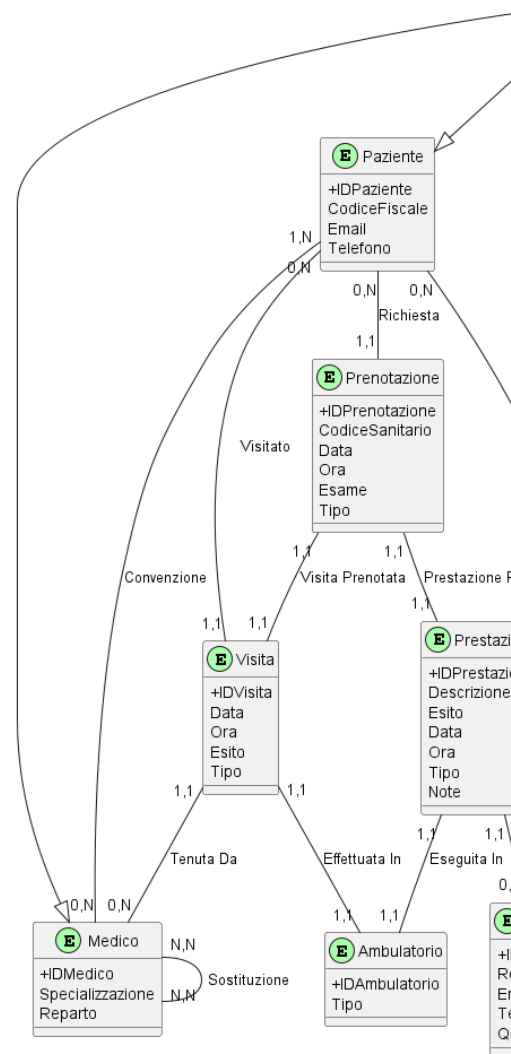
Prenotazione IDPrenotazione, data, ora, richiedente, regimeVisita*, tipo IDPrenotazione

Visita IDVisita, IDPrenotazione, data, ora, tipo, regimeVisita, urgenza, esito, medico, paziente, ambulatorio IDVisita

Prestazione IDPrestazione, IDPrenotazione, data, ora, tipoPrestazione, fase, infermiere, paziente, ambulatorio, note IDPrestazione

Ambulatorio IDAmbulatorio, tipo IDAmbulatorio

Sostituzione medicoSostituto, medicoSostituito medicoSostituto, medicoSostituito



Lo schema implementato della Basi di Dati, completo è il seguente:

Gerarchia

La gerarchia principale del modello riguarda l'entità **Utente**, da cui derivano **Medico**, **Paziente**, **Infermieri** e **Segreteria**. Questa astrazione consente di modellare in modo efficiente attributi e comportamenti comuni a tutte le tipologie di utenti, evitando ridondanze.